

# Funções: Reuso de Código em JavaScript

## 1. Introdução: Organização e Reuso de Código

Quando começamos a programar, é comum escrever todo o código de forma linear, uma instrução após a outra. No entanto, à medida que os programas crescem, essa abordagem se torna problemática.

**Funções** são blocos de código organizados e nomeados que podem ser executados sempre que necessário. Elas são fundamentais para a organização e reuso de código.

### Vantagens da utilização de funções:

- **Reusabilidade:** Escreva uma vez, use várias vezes
- **Manutenção:** Correções e melhorias em um único lugar
- **Legibilidade:** Código mais organizado e fácil de entender
- **Abstração:** Esconda a complexidade por trás de um nome significativo
- **Testabilidade:** Funções podem ser testadas isoladamente

Imagine que você precisa calcular a área de vários retângulos em seu programa. Sem funções, você repetiria o mesmo cálculo várias vezes. Com funções, você escreve a lógica uma vez e a reutiliza sempre que necessário.

### Exercício 1.1

**Reflexão:** Pense em uma tarefa que você já programou e que repetiu código similar em vários lugares. Como uma função poderia ter ajudado?

---

## 2. Criação de Funções

Em JavaScript, existem várias formas de criar funções. Vamos começar com a sintaxe mais básica: a **declaração de função**.

### Sintaxe básica:



```
function nomeDaFuncao() {  
    // código a ser executado  
}
```

## Exemplos simples:



javascript

*// Função que exibe uma mensagem*

```
function saudacao() {  
    console.log("Olá! Bem-vindo ao JavaScript!");  
}
```

*// Para executar (chamar) a função:*

```
saudacao(); // Saída: Olá! Bem-vindo ao JavaScript!
```



javascript

*// Função que realiza um cálculo*

```
function somarDoisMaisTres() {  
    let resultado = 2 + 3;  
    console.log(resultado);  
}
```

```
somarDoisMaisTres(); // Saída: 5
```

**Importante:** A função só é executada quando você a chama pelo nome seguido de parênteses ().

## Exercício 2.1

Crie uma função chamada `mostrarDataAtual` que exiba no console a data atual usando `new Date()`. Depois, chame essa função.

### 3. Parâmetros e Retorno de Valores

Funções ficam muito mais poderosas quando podem receber dados (parâmetros) e devolver resultados (retorno).

#### Sintaxe com parâmetros:



javascript

```
function nomeDaFuncao(parametro1, parametro2) {  
    // código usando os parâmetros  
}
```

#### Sintaxe com retorno:



javascript

```
function nomeDaFuncao(parametro) {  
    // processamento  
    return resultado;  
}
```

#### Exemplos simples:



javascript

*// Função com um parâmetro*

```
function saudarPessoa(nome) {  
    console.log("Olá, " + nome + "!");  
}
```

`saudarPessoa("Maria");` *// Saída: Olá, Maria!*

`saudarPessoa("João");` *// Saída: Olá, João!*



javascript

*// Função com múltiplos parâmetros*

```
function somar(a, b) {  
    return a + b;  
}
```

`let resultado1 = somar(5, 3);`

`console.log(resultado1);` *// Saída: 8*

`let resultado2 = somar(10, 20);`

`console.log(resultado2);` *// Saída: 30*



javascript

*// Função com parâmetros e retorno*

```
function calcularAreaRetangulo(largura, altura) {  
    let area = largura * altura;  
    return area;  
}
```

`let areaQuarto = calcularAreaRetangulo(4, 5);`

`console.log("Área do quarto: " + areaQuarto + "m²");` *// Saída: Área do quarto: 20m²*

**Importante:** Quando uma função usa `return`, ela retorna um valor que pode ser armazenado em uma variável ou usado em outras operações.

### Exercício 3.1

Crie uma função chamada `calcularMedia` que receba dois números como parâmetros e retorne a média entre eles. Teste a função com diferentes valores.

## 4. Escopo de Variáveis

**Escopo** determina onde uma variável pode ser acessada no código. Em JavaScript, existem dois tipos principais de escopo: **global** e **local**.

### Escopo Global

Variáveis declaradas fora de qualquer função têm escopo global e podem ser acessadas de qualquer lugar do código.



javascript

```
let mensagemGlobal = "Eu sou global!";

function mostrarMensagem() {
  console.log(mensagemGlobal); // Acessa a variável global
}

mostrarMensagem(); // Saída: Eu sou global!
console.log(mensagemGlobal); // Saída: Eu sou global!
```

### Escopo Local

Variáveis declaradas dentro de uma função têm escopo local e só podem ser acessadas dentro dessa função.



javascript

```
function criarMensagem() {  
  let mensagemLocal = "Eu sou local!";  
  console.log(mensagemLocal); // Funciona  
}
```

```
criarMensagem(); // Saída: Eu sou local!  
console.log(mensagemLocal); // ERRO! mensagemLocal não existe aqui
```

## Exemplo completo:



javascript

```
let contador = 0; // Variável global
```

```
function incrementar() {  
  let valorLocal = 1; // Variável local  
  contador = contador + valorLocal;  
  console.log("Dentro da função - contador: " + contador);  
}
```

```
incrementar(); // Saída: Dentro da função - contador: 1
```

```
incrementar(); // Saída: Dentro da função - contador: 2
```

```
console.log("Fora da função - contador: " + contador); // Saída: Fora da função - conta  
// console.log(valorLocal); // Isso causaria erro!
```

## Boas práticas:

- Prefira variáveis locais sempre que possível
- Use variáveis globais apenas quando realmente necessário
- Evite conflitos de nomes entre variáveis globais e locais

## Exercício 4.1

Crie um programa com uma variável global `nome` e uma função `alterarNome` que declare uma variável local também chamada `nome` com valor diferente. Dentro da função, exiba a variável local. Fora da função, exiba a variável global.

---

# 5. Reutilização e Encapsulamento da Lógica

O verdadeiro poder das funções está na capacidade de **reutilizar** código e **encapsular** lógica complexa.

## Reutilização

Quando você precisa executar a mesma tarefa múltiplas vezes, funções evitam duplicação de código.



javascript

```
// Sem função (código duplicado)
console.log("=== Relatório ===");
console.log("Usuário: Maria");
console.log("Idade: 25");
console.log("=====");

console.log("=== Relatório ===");
console.log("Usuário: João");
console.log("Idade: 30");
console.log("=====");

// Com função (código reutilizável)
function exibirRelatorio(nome, idade) {
  console.log("=== Relatório ===");
  console.log("Usuário: " + nome);
  console.log("Idade: " + idade);
  console.log("=====");
}

exibirRelatorio("Maria", 25);
exibirRelatorio("João", 30);
```

## Encapsulamento

Encapsular significa esconder a complexidade de uma operação por trás de um nome simples e significativo.



javascript



*// Função que encapsula lógica de validação*

```
function validarIdade(idade) {  
  if (idade < 0) {  
    return false;  
  }  
  if (idade > 150) {  
    return false;  
  }  
  return true;  
}
```

*// Uso simples, sem precisar saber os detalhes da validação*

```
let idade1 = 25;  
if (validarIdade(idade1)) {  
  console.log("Idade válida!");  
}  
  
let idade2 = -5;  
if (validarIdade(idade2)) {  
  console.log("Idade válida!");  
} else {  
  console.log("Idade inválida!");  
}
```

**Exemplo prático combinando reutilização e encapsulamento:**



javascript

```
function calcularDesconto(preco, percentualDesconto) {  
  let desconto = preco * (percentualDesconto / 100);  
  let precoFinal = preco - desconto;  
  return precoFinal;  
}  
  
function exibirProduto(nome, preco, desconto) {  
  let precoComDesconto = calcularDesconto(preco, desconto);  
  console.log("Produto: " + nome);  
  console.log("Preço original: R$ " + preco);  
  console.log("Preço com desconto: R$ " + precoComDesconto);  
  console.log("---");  
}  
  
exibirProduto("Notebook", 3000, 10);  
exibirProduto("Mouse", 50, 20);  
exibirProduto("Teclado", 150, 15);
```

## Exercício 5.1

Crie uma função `validarEmail` que receba uma string e retorne `true` se ela contiver o caractere "@" e `false` caso contrário. Depois, crie uma função `cadaststrarUsuario` que receba nome e email, use a função `validarEmail` e exiba se o cadastro foi bem-sucedido ou não.

---

## 6. Introdução a Funções Arrow

As **arrow functions** (funções seta) são uma sintaxe mais concisa para escrever funções em JavaScript, introduzida no ES6.

### Sintaxe básica:



javascript

*// Função tradicional*

```
function somar(a, b) {  
  return a + b;  
}
```

*// Arrow function equivalente*

```
const somar = (a, b) => {  
  return a + b;  
};
```

## Sintaxe simplificada:

Quando a função tem apenas uma expressão de retorno, podemos simplificar ainda mais:



javascript

*// Arrow function simplificada (sem chaves e sem return)*

```
const somar = (a, b) => a + b;
```

## Exemplos simples:



javascript

*// Arrow function sem parâmetros*

```
const saudacao = () => {  
  console.log("Olá!");  
};
```

```
saudacao(); // Saída: Olá!
```



javascript

*// Arrow function com um parâmetro (parênteses opcionais)*

```
const dobro = numero => numero * 2;
```

```
console.log(dobro(5)); // Saída: 10
```

```
console.log(dobro(10)); // Saída: 20
```



javascript

*// Arrow function com múltiplos parâmetros*

```
const calcularAreaRetangulo = (largura, altura) => largura * altura;
```

```
let area = calcularAreaRetangulo(5, 3);
```

```
console.log(area); // Saída: 15
```



javascript

*// Arrow function com múltiplas linhas*

```
const verificarMaioridade = idade => {  
  if (idade >= 18) {  
    return "Maior de idade";  
  } else {  
    return "Menor de idade";  
  }  
};
```

```
console.log(verificarMaioridade(20)); // Saída: Maior de idade
```

```
console.log(verificarMaioridade(15)); // Saída: Menor de idade
```

## Quando usar arrow functions:

- Para funções curtas e simples
- Quando você quer uma sintaxe mais concisa

- Em callbacks e funções anônimas (verá isso em estudos futuros)

## Exercício 6.1

Converta as seguintes funções tradicionais para arrow functions:



javascript

```
function quadrado(n) {  
  return n * n;  
}  
  
function concatenar(str1, str2) {  
  return str1 + " " + str2;  
}
```

---

## Exercícios de Fixação

### Exercício Fixação 1 (Fácil)

Crie uma função chamada `parOuImpar` que receba um número como parâmetro e retorne a string "Par" se o número for par ou "Ímpar" se for ímpar. Teste com os números 4, 7 e 10.

### Exercício Fixação 2 (Fácil-Médio)

Crie duas funções:

- `celsiusParaFahrenheit`: recebe temperatura em Celsius e retorna em Fahrenheit (fórmula:  $F = C \times 1.8 + 32$ )
- `fahrenheitParaCelsius`: recebe temperatura em Fahrenheit e retorna em Celsius (fórmula:  $C = (F - 32) / 1.8$ )

Teste ambas as funções com diferentes valores.

### Exercício Fixação 3 (Médio)

Crie uma função `calcularIMC` que receba peso (em kg) e altura (em metros) e retorne o IMC (Índice de Massa Corporal =  $\text{peso} / \text{altura}^2$ ). Depois, crie uma função

classificarIMC que receba o IMC e retorne a classificação:

- Abaixo de 18.5: "Abaixo do peso"
- Entre 18.5 e 24.9: "Peso normal"
- Entre 25 e 29.9: "Sobrepeso"
- 30 ou mais: "Obesidade"

Use as duas funções juntas para exibir o IMC e a classificação de uma pessoa.

### Exercício Fixação 4 (Médio-Difícil)

Crie um sistema de notas com as seguintes funções:

- calcularMedia: recebe três notas e retorna a média
- verificarAprovacao: recebe a média e retorna "Aprovado" se  $\geq 7$ , "Recuperação" se  $\geq 5$  e  $< 7$ , ou "Reprovado" se  $< 5$
- exibirBoletim: recebe nome do aluno e três notas, usa as funções anteriores e exibe um boletim completo com nome, notas, média e situação

Teste o sistema com pelo menos 3 alunos diferentes.

---

## Desafio de Encerramento: Sistema de Gerenciamento de Biblioteca

Crie um sistema completo de gerenciamento de uma pequena biblioteca com as seguintes funcionalidades:

### Requisitos:

1. **Função cadastrarLivro:** Recebe título, autor, ano de publicação e número de páginas. Retorna um objeto com os dados do livro e um ID único (pode usar um contador global).
2. **Função calcularIdadeLivro:** Recebe o ano de publicação e retorna quantos anos o livro tem.
3. **Função classificarTamanho:** Recebe o número de páginas e retorna:
  - "Livro curto" se  $< 200$  páginas
  - "Livro médio" se  $\geq 200$  e  $< 400$  páginas
  - "Livro longo" se  $\geq 400$  páginas
4. **Função validarAnoPublicacao:** Recebe um ano e retorna true se for válido (entre 1000 e ano atual) e false caso contrário.
5. **Função gerarResumoLivro:** Recebe os dados de um livro e retorna uma string formatada com todas as informações do livro, incluindo idade e classificação de tamanho.

6. **Função calcularTempoLeitura:** Recebe número de páginas e velocidade de leitura (páginas por hora) e retorna quantas horas serão necessárias para ler o livro.
7. **Função recomendarLivro:** Recebe a idade do leitor e a idade do livro. Retorna true se o livro tiver menos de 5 anos (lançamento recente) ou mais de 50 anos (clássico), sugerindo que são boas opções.

## Desafio Extra:

- Crie uma função compararLivros que receba dois livros e retorne qual é mais longo e qual é mais antigo
- Use arrow functions em pelo menos 3 das funções criadas
- Implemente um menu simples que exiba um relatório de todos os livros cadastrados

## Exemplo de uso esperado:



javascript

```
let livro1 = cadastrarLivro("1984", "George Orwell", 1949, 328);
let livro2 = cadastrarLivro("O Hobbit", "J.R.R. Tolkien", 1937, 310);

console.log(gerarResumoLivro(livro1));
console.log("Tempo de leitura: " + calcularTempoLeitura(livro1.paginas, 30) + " horas");
console.log("Recomendado: " + (recomendarLivro(25, calcularIdadeLivro(livro1.ano)) ? "Sim" : "Não"));
```

Boa sorte! Este desafio integra todos os conceitos aprendidos: criação de funções, parâmetros, retorno de valores, escopo, reutilização, encapsulamento e arrow functions.